

Muhammad Siddiq

Siddiq Report

 Quick Submit Quick Submit National University of Computer and Emerging Sciences, Islamabad

Document Details

Submission ID

trn:oid::1:3423070975

Submission Date

Nov 24, 2025, 3:05 PM GMT+5

Download Date

Nov 24, 2025, 3:07 PM GMT+5

File Name

LAN_Based_Chating_System_Report.docx

File Size

60.5 KB

8 Pages**1,099 Words****5,999 Characters**

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Final Project Report — Programming Fundamentals

University Name: National University Of Computer And Emerging Sciences.

Department: Department of Computer Science

Course: Programming Fundamentals

Project Title: LAN Based Chatting System

Submitted By: Shaikh Muhammad Sameer (Roll No. 25K-0729) & Muhammad Siddique (Roll No. 25K-0610)

Submitted To: Ms. Kinza Mushtaq

Semester: Fall 2025

Date: 21st November 2025

Abstract

The project is a LAN Based Chatting System. It is a console-based program written in C language. It is useful in settings where there are small networks and frequent communication is required. It allows users to communicate with office members using an IP Address. Fundamental programming concepts such as decision structures, loops, arrays, functions, Structures, and Sockets are applied in this project. It shows how data handling can secure communication and reduce the dependency on 3rd party servers for internal office messaging.

1. Introduction

Chatting within an office is sometimes affected by cyber theft. Offices often need a simple digital solution to share messages quickly and securely over a Local Area Network (LAN). This project provides a basic computerized system where users can log in, add contacts, and instantly send text messages to colleagues using the Command Line Interface (CLI).

We chose to write this in C language to understand how variables, loops, and functions work together in a real application. The main goal was to learn the Windows Sockets API (winsock2.h). This is what lets two computers connect. Instead of using complex graphics, we focused on the backend logic to make the messaging fast and efficient using the standard console.

2. Objectives

- To make a messaging system that has the least chances of cyber-attacks.
- To send and receive messages across a LAN from one laptop to another using IP addresses.
- To store the messages inside the computer so we can read them later if needed.
- To practice programming concepts like loops, arrays, functions, structures, file, and directory handling.
- To provide a simple, user-friendly Console-based interface (CLI) with clear menu options.

3. System Design

System Overview

The system works on a Client-Server mode

Server: The main PC that checks passwords and routes and relays messages.

Client: The app where the user types.

Flow of the program:

Start → Display Main Menu → Input Option (1. Login, 2. Signup) → Server Checks File → If Correct, Show User Menu → Input Friend ID & Message → Server Forwards Message → User Selects Logout → End.

Algorithm

1. Start.
2. Initialize Winsock library through WSStartup to prepare the client or sever for network connections.
3. Open the Main Menu (for client only).
4. Display the start screen giving the user two options: 1. Login or 2. Signup. (client side only)
5. Show input prompts for login. (client side only)
6. Ask the user to enter their 7-character User ID and Password.
7. When user enters these details, send the necessary code to the Server.
8. Format the credentials into a data packet (Code 1 + ID + Password) and send it over the socket connected to the server.
9. If Server says "OK" (through sending a client code), clear screen and show Chat Menu.
10. If the server returns response code '1', use system("cls") to clear the console and verify the session is active.
11. User inputs Friend ID and Message, then presses "Enter", The program waits for the user to type the recipient's ID and the text body.
12. Program adds a Server Code to the message Before sending, the system attaches 'Code 4' to the start of the string so the server knows it is a message request.

13. Server checks if friend is online and forwards the text, The server then looks up the Friend ID in its active client list. If found, it relays the message immediately.
14. The user selects "Logout," and the program closes the socket connection.
15. End.

Input & Output

Input:

- LOGIN UI : Username , Password
- Contact UI : Friend User ID
- Chatting Interface : Message, Enter

Output: Display Sent Message on both user's UI And Friend's UI.

4. Implementation

Language: C Using Libraries

- <winsock2.h>
- <stdlib.h>
- <string.h>
- <direct.h>
- <io.h>
- <stdio.h>

Compiler/IDE: Visual Studio Code(Using MinGW)

Code: GCC

Key Features

- **Secure Login:** It checks User_Details.txt to see if the user exists. calculation
- **CLI Interface:** We used Console to display menu and chat messages.

Code Snippet

```
void Make_Initial_Server_Directory(void){ //upon starting server

    if (_access("D:/PF_Project", 0) != 0) {
```

```
// directory does not exist ? create it

int status = _mkdir("D:/PF_Project");


if (status == 0) {
    printf("Directory created successfully.\n");
} else {
    perror("Error creating directory"); //could also use printf
}
}

if (_access("D:/PF_Project/Server", 0) != 0) {
    // directory does not exist ? create it
    int status = _mkdir("D:/PF_Project/Server");

    if (status == 0) {
        printf("Directory created successfully.\n");
    } else {
        perror("Error creating directory"); //could also use printf
    }
}

if (_access("D:/PF_Project/Server/Users", 0) != 0) {
    // directory does not exist ? create it
    int status = _mkdir("D:/PF_Project/Server/Users");

    if (status == 0) {
        printf("Directory created successfully.\n");
    } else {
```

```

        perror("Error creating directory"); //could also use printf
    }
}

FILE *fptr;

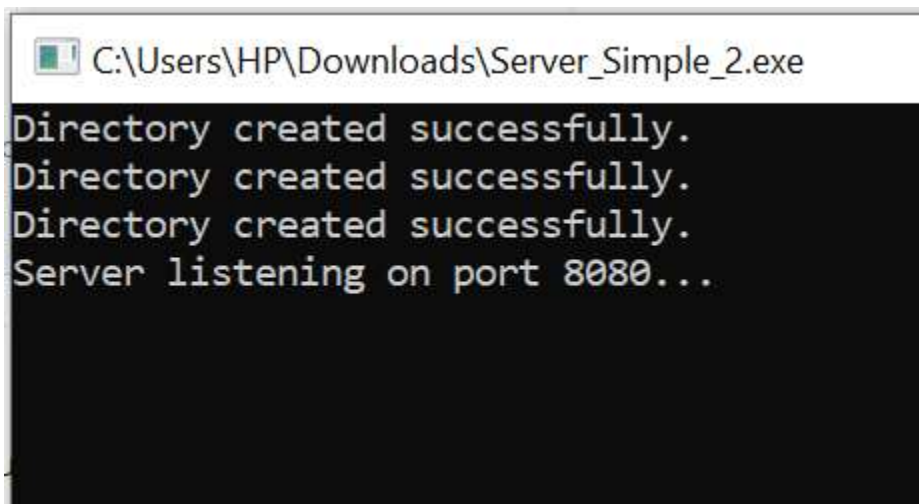
fptr = fopen("D:/PF_Project/Server/Users/User_Details.txt", "a");

fclose(fptr);
}

//Use like this: Make_Initial_Directory();

```

Sample Output



Directory and user details text file created successfully upon running the server script, if directory and file doesn't already exist:

This PC > Local Disk (D:) > PF_Project > Server > Users				
	Name	Date modified	Type	Size
1	User_Details	11/21/2025 2:15 PM	Text Document	0 KB

5. Testing & Results

We tested the project on a home Wi-Fi network.

Test No	Input	Expected Output	Actual Output	Status
---------	-------	-----------------	---------------	--------

1	Login : "25K0729" ,"pass123"	Login Success	"Login Successful" Menu Shown To Select Add New Contact / Chat With Existing Users.	
2	Signup; "25K0610"	Another File Should Be Created	File Updated	
3	Message To Offline User	Error	"User Offline"	

The program worked for all tests. It handled the data traffic efficiently. The console menu was responsive and validated all inputs correctly.

6. Conclusion, Limitations & References

Conclusion

The LAN Based Chatting System shows how to apply Basic Principles of C language to make a real network tool. By using sockets, we managed to send and receive messages using LAN. This increases encryption and minimizes chances of cyber theft. The project helped us understand arrays, loops, structures, and how to implement router logic in server with Winsock networking.

Limitations

- Messages are sent by terminating the connection of user and server then establishing connection between Friend and Server, which takes time.
- Time taking due to repeated connection handshakes.
- We cannot send images right now, only text.

Future Enhancements

- Multiple Client handling at once.

- Once the connection is established there is no need of terminating it to proceed another message.
- Add a scroll bar to the chat history.

References

- Let Us C by Yashavant P. Kanetkar
- <https://www.programiz.com/c-programming>
- <https://www.geeksforgeeks.org/c-programming-language/>